

DISSCO GPU Audio Filter Work: Resume Entry and Mock Interview Q&A

Resume Entry

DISSCO — GPU-Accelerated Audio Filter Pipeline

C++ / CUDA

Contributed to DISSCO (Digital Instrument for Sound Synthesis and Composition), an open-source computer-assisted composition and additive sound-synthesis system (Univ. of Illinois); optimized GPU-accelerated DSP filters in its audio rendering pipeline.

- Implemented a GPU-parallel second-order IIR (biquad) audio filter as a composable 2×2 affine-operator parallel scan over a persistent device-memory arena, with an automated GPU-vs-CPU correctness harness, achieving a measured **4–8×** speedup over a single-threaded CPU baseline (RTX 4090 vs. i9-13900KF).
- Implemented a GPU-parallel all-pass reverb filter as a delay-strided $(D, 2D, 4D, \dots)$ parallel scan with host-orchestrated, fully data-parallel kernel launches, achieving a measured $\sim 4\times$ speedup over CPU.

Mock Interview: Technical Questions

Q: Your resume says you parallelized a second-order IIR filter. By definition, IIR output depends on previous output — $y[n]$ depends on $y[n-1]$. How is that parallelizable at all?

A: The trick is to stop thinking of the recurrence as a scalar update and instead think of it as a linear (affine) map on a small state vector, and to notice that composition of affine maps is associative. For a biquad,

$$\mathbf{s}[n] = \begin{pmatrix} y[n] \\ y[n-1] \end{pmatrix} = \underbrace{\begin{pmatrix} \alpha_1 & \alpha_2 \\ 1 & 0 \end{pmatrix}}_A \mathbf{s}[n-1] + \begin{pmatrix} f[n] \\ 0 \end{pmatrix}, \quad f[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2].$$

Each sample contributes an affine map $(A, \mathbf{b}[n])$ with fixed A and per-sample offset. Two such maps compose as

$$(A_2, \mathbf{b}_2) \circ (A_1, \mathbf{b}_1) = (A_2A_1, A_2\mathbf{b}_1 + \mathbf{b}_2),$$

which is associative but not commutative — exactly the shape a parallel (inclusive) scan needs. Concretely: if map 1 is "identity state feeding forward $f[1]$ " and map 2 is "identity state feeding forward $f[2]$," composing them gives A^2 for the state term and $A \cdot \mathbf{b}_1 + \mathbf{b}_2$ for the offset — i.e. exactly $y[2] = \alpha_1y[1] + \alpha_2y[0] + f[2]$ with $y[1]$ itself already expanded in terms of $y[0]$ and $f[1]$. Running a scan over these per-sample maps and reading off the first component of the composed state at each position gives you $y[n]$ for every n simultaneously, in $O(\log n)$ parallel depth instead of $O(n)$ sequential steps.

Q: What's the difference between a Kogge-Stone/Hillis-Steele scan and a work-efficient (Blelloch) scan, and which did you use?

A: Kogge-Stone and Hillis-Steele are the same construction under two different textbook names: $\log_2 n$ rounds where every active element combines with a neighbor 2^k away, giving $O(n \log n)$ total work but very simple, branch-light code and good behavior at moderate widths. Blelloch does an up-sweep reduction followed by a down-sweep, giving $O(n)$ work but roughly $2 \log_2 n$ sync points and trickier indexing. For a 256-thread block I used a double-buffered Kogge-Stone scan: at that width the extra work is small in absolute terms, and double-buffering (writing each round into a second buffer instead of in place) cuts the required `__syncthreads()` calls from two per round to one, which mattered more in practice than the asymptotic work difference.

Q: Sample buffers can be far longer than one CUDA block can hold. How did you scan something bigger than a single block, or a single kernel launch?

A: Hierarchically, and iteratively rather than with function recursion. Each block scans its own 256 elements and writes its total to a small per-block array; that array of block totals is itself scanned the same way until it fits in one block; then the results are propagated back down by composing each level's now-correct prefix into the level below it. I implemented this as an explicit down-sweep/up-sweep loop over a small fixed-size array of level pointers rather than a recursive function call, mainly so the call structure is easy to reason about on the device side. Because each level shrinks by a factor of 256, even an enormous buffer only needs a handful of levels – an 8-entry stack is already more than enough for any realistic sample count. The first version of this only had one level and silently produced wrong output for buffers with more than $256^2 = 65536$ samples; that surfaced naturally the first time I ran the differential test against a real render more than a couple of seconds long, not from a deliberately constructed edge case, which is part of why I now run that check on every render rather than a one-off suite.

Q: Once the algorithm was correct, what actually limited performance, and how did you find it?

A: Not the scan kernels themselves. For a buffer that fits in the GPU's L2 cache, back-of-envelope memory-bandwidth math put the actual kernel execution time in the low microseconds, but the measured wall-clock time was multiple milliseconds. Counting the CUDA API calls per invocation – three `cudaMallocs`, three `cudaFrees`, three separate blocking `cudaDeviceSynchronize` calls between pipeline stages, two `memcpys` – made it obvious that driver-level allocation and synchronization overhead was the dominant cost, not the algorithm. The fix was a persistent, grow-on-demand device memory arena (allocate once, reuse across calls) and removing the blocking syncs, since kernels issued to the same stream already execute in launch order – the only synchronization actually needed is the final blocking `memcpy` back to the host.

Q: The all-pass filter's recurrence also has a feedback dependency – $y[n]$ depends on $y[n - D]$. Why was parallelizing that comparatively simpler than the biquad case, and why didn't it need the same hierarchical block-then-combine treatment?

A: Because the lag is a single fixed D , not 1. The biquad's natural recurrence step is one sample apart, and a thread block caps out at a few hundred threads, so you're forced to decompose the buffer into blocks, scan within each block, and then combine block results hierarchically. The all-pass filter's doubling scan uses strides $D, 2D, 4D, \dots$ directly, so every round is already a fully data-parallel elementwise operation across the *entire* buffer – there's no natural block boundary to introduce in the first place. That meant the fix was a flat host-driven loop, one kernel launch per round with `gridDim` sized to cover the whole buffer, rather than the two-level scan-then-propagate structure the biquad filter needed. It also meant the round count is $O(\log_2(N/D))$ – for a delay in

the low thousands and a buffer of a few hundred thousand samples, that's on the order of ten kernel launches total, not something that needed the same hierarchical machinery to keep the launch count sane.

Q: Was there a simpler approach you considered, and why didn't you take it?

A: A few, and I think it's worth being explicit about the ones I rejected rather than implying the scan-based design was the only option. The simplest possible GPU version is a straight one-thread port of the CPU's sequential loop – no scan, no matrix reformulation, trivial to write. I didn't ship that because a GPU thread's single-thread throughput is meaningfully worse than a modern CPU core's – lower clock, weaker branch prediction, no out-of-order execution – so it would have been a straightforward regression, not an optimization; the entire point of using a GPU is to spread work across many threads, and a single-thread kernel gives that up entirely. A second option is a brute-force parallel version where each output sample's thread directly sums its own geometrically-decaying series of past inputs, with no scan at all – simpler code, still correct, but $O(n^2)$ total work instead of $O(n \log n)$, which is fine for a few hundred samples and actively worse than the CPU baseline once buffers reach the tens of thousands of samples this system actually processes. A third, more serious option was to leave the per-sample recurrence untouched and parallelize at a coarser grain instead – across tracks or reverb voices rather than within one filter call, since those calls are independent of each other. That's a real, simpler alternative for throughput across a whole render, but it doesn't fit how this code is actually invoked – each call already receives one full buffer to process end-to-end – and more importantly it wouldn't have touched either problem I actually found: a correctness bug is still a correctness bug no matter how many buffers you run through it concurrently, and the fixed per-call allocation overhead is still paid once per buffer regardless of how many buffers run side by side.

More broadly, the order I actually worked in was itself a bias toward the simplest fix that solves the problem: correctness first, since a fast wrong answer is worthless; then the allocation and synchronization fix, which is a small, low-risk change with an outsized effect; and only after that did I reach for the full parallel-scan redesign, and only for the filter where the evidence showed the launch configuration – not just overhead – was still the real ceiling. I'd rather ship several small, individually-justified changes than one clever one that's harder to verify.

Q: Step back from the implementation details – what was the actual significance of this work in the context of the whole project?

A: Two things, and I'd rank the correctness side above the performance side. This audio engine renders whole compositions through a command-line tool that composers launch from the GUI, and reverb is applied per track through exactly the filters I worked on. The pre-existing GPU biquad path had a scan bug that only produced wrong output above roughly 65,000 samples – about a second and a half of audio at 44.1kHz. That threshold is invisible in casual testing but gets crossed by essentially every real piece, so the bug wasn't a rare edge case, it was closer to the default case for actual usage. The all-pass filter's buffer overflow is the same category of problem: an out-of-bounds device write that doesn't reliably crash, so a render can complete, produce audio, and still be wrong, with nothing to signal that anything happened. Those are the worst kind of bug to ship, because "it ran and produced sound" looks exactly like success. So the first-order significance is that the GPU acceleration path – a feature meant to make the tool faster – was silently undermining the correctness of its own output for real projects, and fixing that mattered more than any speedup.

Performance matters too, but for a more human reason than the raw multiplier: this tool is launched interactively by a composer who is going to sit and wait for a render before they can hear what they made and decide whether to change something. Removing fixed per-call overhead that was previously eating most of the GPU budget shortens that wait on every single render, not just in a benchmark – it’s latency in someone’s actual creative feedback loop. And past the two filters I touched directly, I tried to leave a reusable pattern behind rather than two one-off fixes: the persistent-arena approach and the differential correctness harness both generalize directly to the other GPU kernels in the same file that I didn’t get to, so the audit methodology itself is arguably worth as much going forward as the two fixes are today.

Q: Walk me through how you validated correctness, especially in an environment where you couldn’t always get a full build to run.

A: I built the correctness check directly into the code path already used for benchmarking, so every render exercises it for free: run the GPU path and a straightforward serial CPU reference on the identical input, then compare outputs sample-by-sample with a small absolute tolerance (scan-based and sequential summation reorder floating-point rounding differently, so exact bit-equality isn’t the right bar). Each run is tagged with a call id and reports the first mismatching sample index and the largest divergence, not just pass/fail – so if something does break, the location of the first bad sample tells you a lot for free. A regression in the multi-level scan, for instance, shows up as divergence starting exactly at a block-boundary sample index, which points straight at the scan logic instead of generic numerical drift. That mattered more than usual here because these bugs never crash or visibly glitch – wrong output from a broken scan still sounds like plausible audio, so eyeballing a render tells you nothing; only a systematic comparison against a trusted reference catches it.

Q: If you had two more weeks on this, what would you do next?

A: Three things, in order of expected payoff. First, pinned (page-locked) host memory for the input/output buffers, since the host–device memcpy is currently pageable and that’s a well-understood, low-risk win. Second, a cooperative-groups persistent kernel for cases where a filter’s recurrence forces many sequentially-dependent kernel launches, using a grid-wide barrier instead of separate launches, to collapse that chain into a single resident kernel where it’s worth the added complexity. Third, a warp-shuffle-based scan for the innermost rounds to cut shared-memory traffic, though I’d only chase that after confirming with a profiler that the scan itself, and not memcpy or launch overhead, is the remaining bottleneck.

Mock Interview: Behavioral Questions

Q: Tell me about a time you found a critical bug in your own design.

A: During a code review pass on the all-pass filter, I noticed the scratch buffer’s own parameter comment said “size: sampleSize,” but the allocation right above the kernel launch sized it to the filter’s delay length instead – normally far smaller than the sample count, so the kernel was writing past the end of that allocation on essentially every real call. Nothing had crashed, which is the uncomfortable part: an out-of-bounds device write doesn’t reliably fault, it just corrupts whatever memory happens to be nearby. While I was in there I found a second bug in the same kernel: a double-buffer whose true final result landed in either of two buffers depending on whether the scan happened to run an even or odd number of rounds, while the code reading the result back

always read a fixed pointer. Neither bug was caught by a test – they were caught by reading the code closely enough to notice the allocation didn’t match its own documentation. I fixed both, and made sure the differential test I already had elsewhere covered this filter too, rather than trusting the fix on inspection alone.

Q: Describe a time an optimization didn’t produce the result you expected.

A: The first version of the biquad filter’s GPU path was already algorithmically parallel, so I expected a large speedup over a single CPU core almost by default. Measured end-to-end, it was only about $3\times$ – underwhelming for an RTX 4090 against one core of a 13900KF on a workload that should be heavily memory-bandwidth-bound in the GPU’s favor. Rather than assume the algorithm needed more work, I counted what the code was actually doing per call: fresh `cudaMalloc/cudaFree` every invocation, three blocking synchronization points, all for a buffer that fits comfortably in the GPU’s L2 cache. That pointed at fixed per-call driver overhead, not compute, as the real ceiling. Fixing that – a persistent reusable allocation and removing unnecessary syncs – took the measured speedup from about $3\times$ to $4\text{--}8\times$ without touching the scan algorithm at all.

Q: How do you decide when to stop optimizing something?

A: When the remaining ideas stop being clear, low-risk wins and start requiring changes I can’t fully verify in my own environment. After the arena/sync fix and the all-pass filter’s multi-kernel redesign, both filters were meaningfully faster than the CPU baseline and passing an automated correctness check on every run. The next round of ideas – pinned memory, a cooperative-groups rewrite for cases with long sequential launch chains, warp-shuffle scans – are real, but each has a worse risk-to-verification ratio than what I’d already shipped, especially given I couldn’t compile locally to sanity-check them myself. That’s a call I make jointly with whoever owns the timeline: I lay out the remaining techniques, what each would plausibly buy, and what it would cost to verify safely, and let that trade-off decide rather than continuing to tune code without a specific target in mind.

Q: Tell me about a time you had to be upfront about a limitation in your own working process, not just in the code.

A: While reviewing the low-pass comb filter’s parallel implementation, I worked through its coupled recurrence by hand and became suspicious that the carry term passed between processing blocks was wrong – it looked like it was propagating the filter’s output value instead of its internal state, two different quantities that are easy to conflate. But a hand derivation of a genuinely coupled two-index recurrence is the kind of thing that’s easy to get subtly wrong in either direction, and I wasn’t willing to just assert a conclusion from inspection alone. So I said exactly that: flagged it explicitly as “plausible, not confirmed,” explained the specific gap in my own reasoning, and proposed the same automated differential test I’d already built for the other filters rather than either quietly shipping a fix I wasn’t sure was necessary or asserting more confidence than I actually had to sound decisive. It turned out to be a real bug, confirmed by the test, and I fixed it from there. The instinct I’d push back on is that saying “I’m not certain yet” during a review looks weak – it’s what makes the parts you *are* certain of trustworthy, and it’s a lot cheaper than shipping confidently and being wrong.

Q: How do you approach working in math-heavy, unfamiliar code – like DSP recurrences?

A: I don’t trust my first reading of a formula until I’ve derived it myself and checked the details,

because DSP code is exactly the kind of thing where two expressions that look similar encode meaningfully different filters. For the biquad filter, that meant working out by hand that the GPU kernel's state matrix uses $\alpha_1 = -b_{a3}$, $\alpha_2 = -b_{a4}$ – a sign flip relative to the raw feedback coefficients – and confirming that flip was intentional by tracing it back to the CPU reference's `result = ba0*x[n] + ba1*x[n-1] + ba2*x[n-2] - ba3*y[n-1] - ba4*y[n-2]` rather than assuming the GPU code's sign convention was a typo. It's slower up front than trusting the existing code, but it's what let me catch, later, that a different filter's parallel version was carrying the wrong internal state between processing steps – something I wouldn't have been confident diagnosing without having done that derivation work first.

Q: How do you prioritize among several possible performance improvements when you can't test everything yourself?

A: I rank by expected impact and by how confidently I can verify a change before I rank by how interesting the fix is. Counting the actual CUDA API calls per invocation and doing rough bandwidth math told me allocation and synchronization overhead was probably the dominant cost before I changed a single line, so that's what I fixed first – low risk, and easy to reason about correctness-wise. The all-pass filter's full kernel redesign was a bigger, higher-risk change, so I made sure the lower-risk win had landed and was validated by the correctness harness first, then scoped the redesign as a clearly separate step with its own before/after measurement, instead of bundling everything into one change I'd have to debug as a single unit if something came back wrong.